

DSC630Week10.2

November 16, 2025

```
[1]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
[34]: DATASET_LINK='http://files.grouplens.org/datasets/movielens/ml-100k.zip'
```

```
[36]: !wget -nc http://files.grouplens.org/datasets/movielens/ml-100k.zip
!unzip -n ml-100k.zip
```

zsh:1: command not found: wget

Archive: ml-100k.zip

#Loading MovieLens dataset

Loading u.info – The number of users, items, and ratings in the u data set.

```
[37]: overall_stats = pd.read_csv('ml-100k/u.info', header=None)
print("Details of users, items and ratings involved in the loaded movielens_
↳dataset: ",list(overall_stats[0]))
```

Details of users, items and ratings involved in the loaded movielens dataset:

['943 users', '1682 items', '100000 ratings']

Loading u.data – The full u data set, 100000 ratings by 943 users on 1682 items.

Each user has rated at least 20 movies. Users and items are numbered consecutively from 1. The data is randomly ordered. This is a tab separated user id | item id | rating | timestamp.

The time stamps are unix seconds since 1/1/1970 UTC

```
[5]: ## same item id is same as movie id, item id column is renamed as movie id
column_names1 = ['user id','movie id','rating','timestamp']
dataset = pd.read_csv('ml-100k/u.data',
↳sep='\t',header=None,names=column_names1)
dataset.head()
```

```
[5]: user id  movie id  rating  timestamp
0      196      242      3      881250949
```

1	186	302	3	891717742
2	22	377	1	878887116
3	244	51	2	880606923
4	166	346	1	886397596

```
[6]: len(dataset), max(dataset['movie id']),min(dataset['movie id'])
```

```
[6]: (100000, 1682, 1)
```

Loading u.item – Information about the items (movies); this is a tab separated

```
list of
movie id | movie title | release date | video release date |
IMDb URL | unknown | Action | Adventure | Animation |
Children's | Comedy | Crime | Documentary | Drama | Fantasy |
Film-Noir | Horror | Musical | Mystery | Romance | Sci-Fi |
Thriller | War | Western |
The last 19 fields are the genres, a 1 indicates the movie
is of that genre, a 0 indicates it is not; movies can be in
several genres at once.
The movie ids are the ones used in the u.data data set.
```

```
[7]: d = 'movie id | movie title | release date | video release date | IMDb URL |
↳unknown | Action | Adventure | Animation | Children | Comedy | Crime |
↳Documentary | Drama | Fantasy | Film-Noir | Horror | Musical | Mystery |
↳Romance | Sci-Fi | Thriller | War | Western'
column_names2 = d.split(' | ')
print(column_names2)
```

```
['movie id', 'movie title', 'release date', 'video release date', 'IMDb URL',
'unknown', 'Action', 'Adventure', 'Animation', 'Children', 'Comedy', 'Crime',
'Documentary', 'Drama', 'Fantasy', 'Film-Noir', 'Horror', 'Musical', 'Mystery',
'Romance', 'Sci-Fi', 'Thriller', 'War', 'Western']
```

```
[8]: items_dataset = pd.read_csv('ml-100k/u.item',
↳sep='|',header=None,names=column_names2,encoding='latin-1')
items_dataset
```

```
[8]:      movie id      movie title release date \
0          1      Toy Story (1995) 01-Jan-1995
1          2      GoldenEye (1995) 01-Jan-1995
2          3      Four Rooms (1995) 01-Jan-1995
3          4      Get Shorty (1995) 01-Jan-1995
4          5      Copycat (1995) 01-Jan-1995
...      ...
1677      1678      Mat' i syn (1997) 06-Feb-1998
1678      1679      B. Monkey (1998) 06-Feb-1998
1679      1680      Sliding Doors (1998) 01-Jan-1998
1680      1681      You So Crazy (1994) 01-Jan-1994
```

1681 1682 Scream of Stone (Schrei aus Stein) (1991) 08-Mar-1996

	video release date	IMDb URL \
0	NaN	http://us.imdb.com/M/title-exact?Toy%20Story%20...
1	NaN	http://us.imdb.com/M/title-exact?GoldenEye%20(...
2	NaN	http://us.imdb.com/M/title-exact?Four%20Rooms%...
3	NaN	http://us.imdb.com/M/title-exact?Get%20Shorty%...
4	NaN	http://us.imdb.com/M/title-exact?Copycat%20(1995)
...	...	...
1677	NaN	http://us.imdb.com/M/title-exact?Mat%27+i+syn+...
1678	NaN	http://us.imdb.com/M/title-exact?B%2E+Monkey+(...
1679	NaN	http://us.imdb.com/Title?Sliding+Doors+(1998)
1680	NaN	http://us.imdb.com/M/title-exact?You%20So%20Cr...
1681	NaN	http://us.imdb.com/M/title-exact?Schrei%20aus%...

	unknown	Action	Adventure	Animation	Children	...	Fantasy	\
0	0	0	0	1	1	...	0	
1	0	1	1	0	0	...	0	
2	0	0	0	0	0	...	0	
3	0	1	0	0	0	...	0	
4	0	0	0	0	0	...	0	
...	...	...	...	...	...	...		
1677	0	0	0	0	0	...	0	
1678	0	0	0	0	0	...	0	
1679	0	0	0	0	0	...	0	
1680	0	0	0	0	0	...	0	
1681	0	0	0	0	0	...	0	

	Film-Noir	Horror	Musical	Mystery	Romance	Sci-Fi	Thriller	War	\
0	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	1	0	
2	0	0	0	0	0	0	1	0	
3	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	1	0	
...	...	...	...	...	...	...	...	...	
1677	0	0	0	0	0	0	0	0	
1678	0	0	0	0	1	0	1	0	
1679	0	0	0	0	1	0	0	0	
1680	0	0	0	0	0	0	0	0	
1681	0	0	0	0	0	0	0	0	

	Western
0	0
1	0
2	0
3	0
4	0

```
...      ...
1677      0
1678      0
1679      0
1680      0
1681      0
```

[1682 rows x 24 columns]

```
[9]: movie_dataset = items_dataset[['movie id', 'movie title']]
movie_dataset.head()
```

```
[9]:   movie id      movie title
0         1  Toy Story (1995)
1         2  GoldenEye (1995)
2         3  Four Rooms (1995)
3         4  Get Shorty (1995)
4         5   Copycat (1995)
```

Looking at length of original items_dataset and length of unique combination of rows in items_dataset after removing movie id column

```
[10]: ## looking at length of original items_dataset and length of unique combination
      ↳ of rows in items_dataset after removing movie id column
len(items_dataset.groupby(by=column_names2[1:]), len(items_dataset))
```

```
[10]: (1664, 1682)
```

We can see there are 18 extra movie id's for already mapped movie title and the same duplicate movie id is assigned to the user in the user-item dataset.

#Merging required datasets

```
[11]: merged_dataset = pd.merge(dataset, movie_dataset, how='inner', on='movie id')
merged_dataset.head()
```

```
[11]:   user id  movie id  rating  timestamp      movie title
0      196      242       3  881250949      Kolya (1996)
1      186      302       3  891717742  L.A. Confidential (1997)
2       22      377       1  878887116   Heavyweights (1994)
3      244       51       2  880606923  Legends of the Fall (1994)
4      166      346       1  886397596   Jackie Brown (1997)
```

A dataset is created from the existing merged dataset by grouping the unique user id and movie title combination and the ratings by a user to the same movie in different instances (timestamps) are averaged and stored in the new dataset.

Example of a multiple rating scenario by an user to a specific movie:

```
[12]: merged_dataset[(merged_dataset['movie title'] == 'Chasing Amy (1997)') &
↳(merged_dataset['user id'] == 894)]
```

```
[12]:      user id  movie id  rating  timestamp      movie title
62716      894      246      4  882404137  Chasing Amy (1997)
90596      894      268      3  879896041  Chasing Amy (1997)
```

0.1 Creating a final refined dataset with unique user id, movie name combination and their ratings:

```
[13]: refined_dataset = merged_dataset.groupby(by=['user id','movie title'],
↳as_index=False).agg({"rating":"mean"})

refined_dataset.head()
```

```
[13]:      user id      movie title  rating
0         1      101 Dalmatians (1996)    2.0
1         1      12 Angry Men (1957)    5.0
2         1  20,000 Leagues Under the Sea (1954)    3.0
3         1    2001: A Space Odyssey (1968)    4.0
4         1      Abyss, The (1989)    3.0
```

0.2 Creating lists for unique user id's and movie names:

```
[14]: #list of all users
unique_users = refined_dataset['user id'].unique()
#creating a list of all movie names in it
unique_movies = refined_dataset['movie title'].unique()
len(unique_movies),len(unique_users)
```

```
[14]: (1664, 943)
```

0.3 Converting user id, movie name column of refined dataset to respective lists:

```
[15]: users_list = refined_dataset['user id'].tolist()
movie_list = refined_dataset['movie title'].tolist()
len(users_list),len(movie_list)
```

```
[15]: (99693, 99693)
```

0.4 Extracting ratings into a list:

```
[38]: ratings_list = refined_dataset['rating'].tolist()
len(ratings_list)
```

```
[38]: 99693
```

0.5 Creating a dictionary to map movie name to their corresponding index in the unique movie name list

```
[39]: movies_dict = {unique_movies[i] : i for i in range(len(unique_movies))}
      print(len(movies_dict))
```

1664

0.6 Creating a Utility matrix with rows as movies, columns as users, to make the refined dataframe compatible for SVD operations

```
[40]: ## creating a utility matrix for the available data

      ## Creating an empty array with (number of rows = number of movies) and (number
      ## of columns = number of users) rows as movies, columns as users

      utility_matrix = np.asarray([[np.nan for j in range(len(unique_users))] for i
      ## in range(len(unique_movies))])
      print("Shape of Utility matrix: ",utility_matrix.shape)

      for i in range(len(ratings_list)):

          ## ith entry in users list and subtract 1 to get the index, we do the same
          ## for movies but we already defined a dictionary to get the index.
          utility_matrix[movies_dict[movie_list[i]]][users_list[i]-1] = ratings_list[i]

      utility_matrix
```

Shape of Utility matrix: (1664, 943)

```
[40]: array([[ 2., nan, nan, ..., nan, nan, nan],
      [ 5., nan, nan, ..., nan, nan, nan],
      [ 3., nan, nan, ..., nan, nan, nan],
      ...,
      [nan, nan, nan, ..., nan, nan, nan],
      [nan, nan, nan, ..., nan, nan, nan],
      [nan, nan, nan, ..., nan, nan, nan]])
```

0.7 Normalizing the utility matrix across movies column

```
[41]: mask = np.isnan(utility_matrix)
      masked_arr = np.ma.masked_array(utility_matrix, mask)
      temp_mask = masked_arr.T
      rating_means = np.mean(temp_mask, axis=0)

      filled_matrix = temp_mask.filled(rating_means)
      filled_matrix = filled_matrix.T
      filled_matrix = filled_matrix - rating_means.data[:,np.newaxis]
```

```
[42]: filled_matrix = filled_matrix.T / np.sqrt(len(movies_dict)-1)
filled_matrix
```

```
[42]: array([[ -0.02227217,  0.01608636, -0.01226094, ...,  0.          ,
           0.          ,  0.          ],
       [ 0.          ,  0.          ,  0.          , ...,  0.          ,
           0.          ,  0.          ],
       [ 0.          ,  0.          ,  0.          , ...,  0.          ,
           0.          ,  0.          ],
       ...,
       [ 0.          ,  0.          ,  0.          , ...,  0.          ,
           0.          ,  0.          ],
       [ 0.          ,  0.          ,  0.          , ...,  0.          ,
           0.          ,  0.          ],
       [ 0.          ,  0.          ,  0.          , ...,  0.          ,
           0.          ,  0.          ]])
```

Mean values across movies columns are extracted.

Nan's of utility matrix are imputed with mean values extracted.

Later the utility matrix has been normalized across movies to get all ratings to a standard/normal scale.

```
[43]: filled_matrix.shape
```

```
[43]: (943, 1664)
```

0.8 Computing SVD (Singular Value Decomposition) of Utility matrix

```
[44]: ## Computing the SVD of the input matrix

U, S, V = np.linalg.svd(filled_matrix)
```

0.9 Creating a list of Case insensitive movie names for further use

```
[45]: case_insensitive_movies_list = [i.lower() for i in unique_movies]
```

0.10 Defining a function to calculate the cosine similarity on the given dataframe and extracting requesting number of closely matched movie indices with the help of numpy einsum which valuates the Einstein summation convention on the operands.

```
[46]: #Function to calculate the cosine similarity (sorting by most similar and
↪returning the top N)
def top_cosine_similarity(data, movie_id, top_n=10):
    index = movie_id
    movie_row = data[index, :]
    magnitude = np.sqrt(np.einsum('ij, ij -> i', data, data))
```

```

similarity = np.dot(movie_row, data.T) / (magnitude[index] * magnitude)
sort_indexes = np.argsort(-similarity)
return sort_indexes[:top_n]

```

0.11 Defining a function to get similar movies for the given movie name

```

[47]: #k-principal components to represent movies, movie_id to find recommendations, top_n print n results
def get_similar_movies(movie_name, top_n, k = 50):
    # k = 50
    # movie_id = 1
    # top_n = 10

    sliced = V.T[:, :k] # representative data
    movie_id = movies_dict[movie_name]
    indexes = top_cosine_similarity(sliced, movie_id, top_n)
    print(" ")
    print("Top", top_n, "movies which are very much similar to the",
    Movie-, movie_name, "are: ")
    print(" ")
    for i in indexes[1:]:
        print(unique_movies[i])

```

In SVD decomposition, Where A is a $m \times n$ utility matrix, U is a $m \times r$ orthogonal left singular matrix, which represents the relationship between users and latent factors, S is a $r \times r$ diagonal matrix, which describes the strength of each latent factor and V is a $r \times n$ diagonal right singular matrix, which indicates the similarity between items and latent factors. The latent factors here are the characteristics of the items, for example, the genre of the music. The SVD decreases the dimension of the utility matrix A by extracting its latent factors. It maps each user and each item into a r -dimensional latent space. This mapping facilitates a clear representation of relationships between users and items.

Dynamically suggesting movie name from the existing movie corpus we have, based on the user input using try and except architecture.

Defining a function which outputs movie names as suggestion when the user mis spells the movie name. **User might have intended to type any of these movie names.**

```

[48]: # function which takes input and returns suggestions for the user
def get_possible_movies(movie):

    temp = ''
    possible_movies = case_insensitive_movies_list.copy()
    for i in movie :
        out = []
        temp += i
        for j in possible_movies:

```



```

        if temp in j:
            out.append(j)
        if len(out) == 0:
            return possible_movies
        out.sort()
        possible_movies = out.copy()

    return possible_movies

```

This function provides user with **movie name suggestions if movie name is mis-spelled or Recommends similar movies to the input movie** if the movie name is valid.

```

[49]: class invalid(Exception):
        pass

    def recommender():

        try:

            movie_name = input("Enter the Movie name: ")
            movie_name_lower = movie_name.lower()
            if movie_name_lower not in case_insensitive_movies_list :
                raise invalid
            else :
                # movies_list[case_insensitive_country_names.index(movie_name_lower)]
                num_recom = int(input("Enter Number of movie recommendations needed: "))
                get_similar_movies(unique_movies[case_insensitive_movies_list.
↪index(movie_name_lower)],num_recom)

        except invalid:

            possible_movies = get_possible_movies(movie_name_lower)

            if len(possible_movies) == len(unique_movies) :
                print("Movie name entered is does not exist in the list ")
            else :
                indices = [case_insensitive_movies_list.index(i) for i in
↪possible_movies]
                print("Entered Movie name is not matching with any movie from the
↪dataset . Please check the below suggestions :\n",[unique_movies[i] for i in
↪indices])
                print("")
                recommender()

```

```

[ ]: recommender()

```

0.12 Conclusion:

The above built recommender system can suggest movie names similar to the given movie name. The same architecture can be built to get similar users for the given user id and then suggesting movies from those user id's (this would be item-item collaborative filtering). This can be done just by extracting latent factors of U matrix (which corresponds to Users) instead V matrix (which corresponds to Movies) as we did. But a better recommender system can be built only if we could change the axis of normalization of utility matrix to userid from movies.

[]: